

Guia do trabalho 1

📅 Created Date	September 18, 2026
☰ Tags	Programação prolog trabalho
➤ learning topics	</> <u>Paradigmas de programação</u>

Guia do Trabalho — Paradigmas de Programação

Disciplina: Paradigmas de Programação

Curso: Ciência da Computação — UFF Rio das Ostras

Tema: Analisador sintático da linguagem de Lógica Proposicional usando FLEX e YACC

1. O que o trabalho pede?

O trabalho consiste em desenvolver e modificar um analisador para a linguagem de Lógica Proposicional.

O projeto pode ser dividido em 5 fases:

1. Entender os arquivos fornecidos pela professora
 2. Compilar a versão original
 3. Testar os arquivos `t1.log` até `t8.log`
 4. Adicionar o operador `<->`
 5. Transformar o analisador em uma calculadora de valores lógicos
-

2. Arquivos fornecidos

Os principais arquivos são:

- `LogProp.pdf` → especificação da linguagem
 - `log.lex` → especificação do analisador léxico para FLEX
 - `log.y` → especificação do analisador sintático para YACC
 - `prog1.log` → programa de teste
 - `t1.log` até `t8.log` → testes fornecidos pela professora
-

3. Como funciona o projeto?

O funcionamento geral é:

```
log.lex
  ↓
FLEX
  ↓
lex.yy.c

log.y
  ↓
YACC
  ↓
y.tab.c + y.tab.h

lex.yy.c + y.tab.c
  ↓
gcc
  ↓
programa "log"
```

O FLEX cuida da parte **léxica**, identificando os símbolos.

O YACC cuida da parte **sintática**, verificando se os símbolos formam uma fórmula válida.

4. Entender a linguagem

De acordo com a especificação `LogProp.pdf`, a linguagem possui os seguintes elementos:

Operadores

~	NÃO
&	E
	OU
->	SE...ENTÃO
<->	SE E SOMENTE SE

Parênteses

(
)

Símbolos de sentença

a
a1
a2
a3
...

Uma fórmula bem formada pode ser construída pelas seguintes regras:

$(\sim\alpha)$

 $(\alpha \ \& \ \beta)$

 $(\alpha \ | \ \beta)$

 $(\alpha \ \rightarrow \ \beta)$

 $(\alpha \ \leftrightarrow \ \beta)$

Além disso, um símbolo de sentença sozinho também é uma fórmula bem formada.

Exemplos:

```
a
(~a)
((~a) <-> a1)
```

Exemplos que NÃO são fórmulas bem formadas:

```
(a)
~a
((a<->a)&(~b))
```

5. FASE 1 — Entender o **log.lex**

O **log.lex** é o analisador léxico.

Podemos imaginar que ele transforma:

```
((a->a)&(~b))
```

em algo parecido com:

```
(
(
a
->
a
)
&
(
~
b
)
)
```

e depois identifica cada elemento como um token.

Por exemplo:

Símbolo	Token
~	NAO
&	E
,	,
->	SE
(	ABRE
)	FECHA
a, a1, a2 ...	SIMB

A expressão:

a1

é reconhecida como um símbolo de sentença.

6. FASE 2 — Entender o log.y

O log.y contém a gramática da linguagem.

A regra principal atualmente possui uma estrutura semelhante a:

```
form → atom
      | ( ~ form )
      | ( form & form )
      | ( form | form )
      | ( form -> form )
```

Isso significa que uma form pode ser:

1. Um átomo
2. Uma negação
3. Uma operação AND
4. Uma operação OR
5. Uma implicação

Por exemplo:

```
(~a)
```

pode ser interpretado como:

```
(  
  ~  
  a  
)
```

O FLEX identifica:

```
ABRE  
NAO  
SIMB  
FECHA
```

e o YACC verifica se essa sequência corresponde à regra:

```
ABRE NAO form FECHA
```

7. FASE 3 — Compilar a versão original

Antes de modificar qualquer coisa, devemos fazer a versão original funcionar.

7.1 Gerar o analisador léxico

No terminal:

```
flex log.lex
```

Isso gera:

```
lex.yy.c
```

Não devemos compilar ainda.

7.2 Gerar o analisador sintático

Execute:

```
yacc -d log.y
```

Isso gera:

```
y.tab.c  
y.tab.h
```

7.3 Compilar

Execute:

```
gcc y.tab.c lex.yy.c -o log
```

Isso gera o executável:

```
log
```

No Linux, normalmente executaremos:

```
./log t1.log
```

8. FASE 4 — Testar os arquivos fornecidos

Agora devemos testar:

```
./log t1.log  
./log t2.log  
./log t3.log  
./log t4.log  
./log t5.log  
./log t6.log  
./log t7.log  
./log t8.log
```

A ideia é registrar quais fórmulas são aceitas e quais são rejeitadas.

t1.log

Conteúdo:

```
a
```

Deve ser válido.

t2.log

Conteúdo:

```
(~a)
```

Deve ser válido.

t3.log

Conteúdo:

```
((~a) -> a1)
```

Deve ser válido.

t4.log

Conteúdo:

```
(A)
```

Deve ser inválido.

O símbolo **A** maiúsculo não corresponde ao padrão dos símbolos de sentença especificados.

t5.log

Conteúdo:

```
(~a)
```

Deve ser válido.

t6.log

Conteúdo:

```
((a->a)&(~b))
```

Deve ser válido.

A estrutura é:

```
(  
  (a -> a)  
  &  
  (~b)  
)
```

t7.log

Conteúdo:

```
((a->a)&((~b))
```

Deve ser inválido porque os parênteses não estão balanceados de acordo com a gramática.

t8.log

Conteúdo:

```
((a1<->a2)&(~a))
```

Esse teste é especialmente importante.

A especificação da linguagem possui o operador:

```
<->
```

porém a implementação original ainda não o reconhece.

Esse é justamente um dos pontos que precisamos alterar.

9. Tabela para registrar os testes

Podemos criar no Notion uma tabela como:

Arquivo	Fórmula	Versão original	Depois da alteração
t1	a	✓	✓
t2	(~a)	✓	✓
t3	((~a)->a1)	✓	✓
t4	(A)	✗	✗
t5	(~a)	✓	✓

Arquivo	Fórmula	Versão original	Depois da alteração
t6	<code>((a->a)&(~b))</code>	✓	✓
t7	<code>((a->a)&((~b))</code>	✗	✗
t8	<code>((a1<->a2)&(~a))</code>	✗	✓

10. FASE 5 — Adicionar o operador `<->`

Agora começa a primeira alteração solicitada pela professora.

Precisamos modificar:

```
log.lex
```

e:

```
log.y
```

10.1 Alterar o `log.lex`

Precisamos adicionar uma regra para reconhecer:

```
<->
```

Por exemplo:

```
"<->" { return SESE; }
```

`SESE` é apenas um nome sugerido para o token.

O importante é que o mesmo nome seja utilizado no `log.y`.

10.2 Alterar o `log.y`

Na declaração dos tokens, adicionar o novo token.

Por exemplo:

```
%token NAO E OU SE SESE ABRE FECHA
```

Depois adicionar uma regra para o novo operador:

```
| ABRE form SESE form FECHA
```

Isso representa:

```
( form <-> form )
```

11. Recompilar depois da alteração

Sempre que modificarmos `log.lex` ou `log.y`, precisamos gerar os arquivos novamente.

```
flex log.lex  
yacc -d log.y  
gcc y.tab.c lex.yy.c -o log
```

Depois testar novamente:

```
./log t8.log
```

Agora o `t8.log` deve ser reconhecido.

12. FASE 6 — Criar a calculadora lógica

Depois de fazer o analisador sintático funcionar, começa a segunda parte importante do trabalho.

Agora o programa não deve apenas dizer:

"A fórmula está correta."

Ele também deverá calcular:

"Qual é o valor lógico dessa fórmula?"

Nesta etapa, a professora determina que serão utilizados apenas:

v
f

onde:

v = verdadeiro
f = falso

Exemplos:

v

(~f)

((~v)<->f)

13. Regras da calculadora

Precisamos implementar os valores dos operadores.

Negação

~v = f
~f = v

E

```
v & v = v  
v & f = f  
f & v = f  
f & f = f
```

OU

```
v | v = v  
v | f = v  
f | v = v  
f | f = f
```

Implicação

```
v -> v = v  
v -> f = f  
f -> v = v  
f -> f = v
```

Bicondicional

```
v <-> v = v  
v <-> f = f  
f <-> v = f  
f <-> f = v
```

14. Como a calculadora deve funcionar

Por exemplo:

```
(~f)
```

Primeiro:

```
f
↓
Falso
```

Depois:

```
~f
↓
Verdadeiro
```

Resultado:

```
v
```

Outro exemplo:

```
((~v)<->f)
```

Primeiro:

```
v
↓
Verdadeiro
```

Depois:

```
~v
↓
Falso
```

Então:

```
f <-> f
↓
Verdadeiro
```

Resultado:

```
v
```

15. Como implementar isso no YACC

O `log.y` atualmente está preocupado principalmente em reconhecer a estrutura. Na calculadora, precisamos fazer cada regra também produzir um valor lógico. A ideia é:

```
form
↓
valor lógico
```

Por exemplo:

```
v
↓
VERDADEIRO
```

e:

```
f
↓
FALSO
```

Então uma regra como:

```
(~form)
```

deve calcular o valor de `form` e depois inverter esse valor.

Da mesma maneira:

```
(form & form)
```

deve calcular os dois lados e aplicar AND.

16. Divisão do trabalho entre os 4 integrantes

Integrante 1 — Você

Responsável por:

- Coordenação geral
- Entender a gramática
- Trabalhar principalmente no `log.y`
- Adicionar `<->` na gramática
- Integrar as partes do projeto
- Fazer a versão final funcionar

Também deve entender:

- Como FLEX e YACC se comunicam
 - Como os valores lógicos serão propagados pelo YACC
-

Integrante 2 — Analisador léxico

Responsável por:

log.lex

Tarefas:

- Entender os tokens
- Conferir os símbolos da linguagem
- Adicionar `<->`
- Verificar `v` e `f`
- Verificar símbolos inválidos
- Testar o reconhecimento dos tokens

Pode criar uma tabela:

Lexema	Token
<code>~</code>	<code>NAO</code>
<code>&</code>	<code>E</code>
<code>,</code>	<code>,</code>
<code>-></code>	<code>SE</code>
<code><-></code>	<code>SESE</code>
<code>(</code>	<code>ABRE</code>
<code>)</code>	<code>FECHA</code>
<code>a</code> , <code>a1</code> , <code>a2</code> ...	<code>SIMB</code>

Integrante 3 — Testes

Responsável por:

```
t1.log  
t2.log  
t3.log  
t4.log  
t5.log  
t6.log
```

```
t7.log  
t8.log
```

Tarefas:

- Executar todos os testes
- Registrar os resultados
- Identificar casos válidos e inválidos
- Criar testes adicionais
- Testar o `<->`
- Testar a calculadora

Testes adicionais sugeridos:

```
v  
f  
(~v)  
(~f)  
(v&f)  
(v|f)  
(v->f)  
(v<->f)
```

E testes inválidos:

```
(v)  
~v  
(v&)  
(v<->)  
((v)
```

Integrante 4 — Calculadora

Responsável por:

- Pesquisar como retornar valores pelo YACC

- Definir como representar `v` e `f`
- Implementar as operações lógicas
- Testar expressões simples
- Testar expressões compostas

Deve conseguir explicar manualmente exemplos como:

```
((~v)<->f)
```

Resultado:

```
v
```

17. Ordem correta para desenvolver

É importante NÃO tentar fazer tudo ao mesmo tempo.

Sigam esta ordem:

Etapa A — Versão original

Fazer:

```
flex log.lex  
yacc -d log.y  
gcc y.tab.c lex.yy.c -o log
```

funcionar.

Etapa B — Testes originais

Executar:

```
./log t1.log  
./log t2.log  
./log t3.log  
./log t4.log  
./log t5.log  
./log t6.log  
./log t7.log  
./log t8.log
```

Registrar os resultados.

Etapa C — Adicionar <-->

Alterar:

```
log.lex
```

e:

```
log.y
```

Etapa D — Testar novamente

Regenerar:

```
flex log.lex  
yacc -d log.y  
gcc y.tab.c lex.yy.c -o log
```

E testar:

```
./log t8.log
```

Etapa E — Implementar a calculadora

Somente depois que o parser estiver funcionando corretamente.

Implementar:

```
v
f
~
&
|
->
<->
```

Etapa F — Testes finais

Criar vários casos para verificar:

- Fórmulas válidas
- Fórmulas inválidas
- Negação
- AND
- OR
- Implicação
- Bicondicional
- Fórmulas compostas

18. Organização recomendada do projeto

Uma estrutura simples seria:

```
trabalho-logprop/
|
├─ original/
|   └─ log.lex
|   └─ log.y
```

```
|   ├── prog1.log  
|   └── t1.log ... t8.log  
|  
├── projeto/  
|   ├── log.lex  
|   ├── log.y  
|   ├── prog1.log  
|   └── t1.log ... t8.log  
└── README.md
```

19. Usar Git

Como são quatro pessoas, é recomendado utilizar Git.

Primeiro:

```
git init  
git add .  
git commit -m "Versão original da professora"
```

Depois cada integrante pode trabalhar em uma branch.

Por exemplo:

```
git checkout -b operador-bicondicional
```

ou:

```
git checkout -b calculadora
```

ou:

```
git checkout -b testes
```

Isso reduz bastante o risco de alguém sobrescrever o trabalho de outra pessoa.

20. Checklist final

Entendimento

- ☐ Todos entenderam a especificação do `LogProp.pdf`
- ☐ Todos entenderam o que é um token
- ☐ Todos entenderam o papel do FLEX
- ☐ Todos entenderam o papel do YACC
- ☐ Todos entenderam a gramática

Versão original

- ☐ `flex log.lex` funciona
- ☐ `yacc -d log.y` funciona
- ☐ `gcc y.tab.c lex.yy.c -o log` funciona
- ☐ `t1.log` testado
- ☐ `t2.log` testado
- ☐ `t3.log` testado
- ☐ `t4.log` testado
- ☐ `t5.log` testado
- ☐ `t6.log` testado
- ☐ `t7.log` testado
- ☐ `t8.log` testado

Operador `<->`

- ☐ `<->` adicionado ao `log.lex`
- ☐ Token adicionado ao `log.y`
- ☐ Regra sintática adicionada

- ☐ Projeto recompilado
- ☐ `t8.log` testado novamente

Calculadora

- ☐ `v` funciona
- ☐ `f` funciona
- ☐ `~` funciona
- ☐ `&` funciona
- ☐ `|` funciona
- ☐ `>` funciona
- ☐ `<->` funciona
- ☐ Fórmulas compostas funcionam
- ☐ Casos inválidos continuam sendo rejeitados

Entrega

- ☐ Código organizado
- ☐ Todos os integrantes entendem o código
- ☐ Testes documentados
- ☐ Código final compilando
- ☐ README/documentação preparado, se solicitado pela professora